

GPU-accelerated single-cell and spatial transcriptomics on NVIDIA DGX H100: a systematic benchmark of speed, scalability, and biological concordance

Abstract

Single-cell RNA sequencing (scRNA-seq) datasets now routinely exceed one million cells, and spatial transcriptomics platforms such as Visium HD generate hundreds of thousands of measurement bins per tissue section, placing severe computational demands on analysis frameworks that remain predominantly CPU-bound. We present a systematic benchmark comparing Scanpy (CPU, 100 cores) against rapids-singlecell (GPU, 1-8 NVIDIA H100 80 GB) on the 1.3-million mouse brain cell dataset, evaluating speed, multi-GPU scalability, memory efficiency, and biological concordance across five dataset sizes (10k-1.3M cells) with five independent repeats. The GPU pipeline achieved up to 120-fold end-to-end speedup at 1.3M cells (435 s vs 52,056 s on CPU), reducing a 14.5-hour analysis to 7.3 minutes.

Biological concordance was high: HVG selection was identical (Jaccard = 1.0), PCA loadings perfectly correlated (Spearman $|\rho| = 1.0$), and Leiden clustering concordance ranged from ARI = 0.908 to 0.963. Multi-GPU scaling was sublinear due to CPU preprocessing dominance. A memory-optimised pipeline processed 11.9 million cells on a single DGX H100 node, with CPU RAM as the binding constraint (GPU VRAM at 7.6% of capacity). A differential expression benchmark at 3.4M cells showed pseudo-bulk aggregation 44x faster than cell-level t-test. We extended the benchmark to spatial transcriptomics on three 10x Visium platforms, achieving up to 51.6x end-to-end speedup (Visium HD 8 um) and 3,272x for co-occurrence analysis, with near-perfect spatial autocorrelation concordance (Spearman $\rho \geq 0.9995$). These results identify CPU-side preprocessing as the primary bottleneck and provide practical guidance for deploying GPU-accelerated workflows at atlas scale.

Keywords: GPU computing, single-cell RNA-seq, spatial transcriptomics, benchmarking, rapids-singlecell

1 Introduction

Single-cell RNA sequencing (scRNA-seq) has become the standard technology for dissecting cellular heterogeneity, with atlas-scale projects now generating datasets of hundreds of thousands to millions of cells [1], [2]. The dominant analysis framework, Scanpy [3], runs on CPU and relies on NumPy, SciPy, and scikit-learn for linear algebra and graph operations. A standard pipeline encompassing quality control, normalisation, PCA, Leiden clustering [4], UMAP [5], and differential expression (DE) testing can require hours to days on a multi-core workstation at atlas scale. In parallel, spatial transcriptomics platforms such as Visium HD generate hundreds of thousands of measurement bins per tissue section, and spatial analysis pipelines built on Squidpy [6] add computationally intensive operations (spatial autocorrelation, co-occurrence analysis) on top of the standard expression workflow.

GPU-accelerated alternatives have emerged to address this challenge. The RAPIDS ecosystem [7] provides GPU-native data science primitives, and rapids-singlecell [8] offers a near-drop-in replacement for Scanpy on NVIDIA GPUs. Prior work has demonstrated substantial GPU speedups for genomic analyses [9], [10] and proposed GPU frameworks for datasets exceeding ten million cells [11]. However, most benchmarks report headline speedups without systematically examining per-step performance variation, multi-GPU scaling behaviour, numerical concordance with CPU baselines, and practical memory limits at extreme scale. Gardner et al. [12] recently evaluated accuracy-performance trade-offs, but their study was limited to a single GPU.

Here we present a comprehensive benchmark addressing five questions: (1) per-step and end-to-end GPU speedup; (2) multi-GPU scaling from 1 to 8 H100 GPUs; (3) biological

concordance between CPU and GPU outputs; (4) maximum dataset capacity on a single DGX H100 node; and (5) generalisability to spatial transcriptomics.

2 Data and Methods

2.1 Single-cell RNA-seq benchmark

All experiments used the 10x Genomics 1.3-million mouse brain cell dataset (E18) [13]. Reproducible subsamples of 10k, 50k, 100k, and 500k cells were created by uniform random sampling (seed = 42). Both pipelines implemented the standard best-practices workflow [14], [15] with identical parameters: QC filtering, library-size normalisation, HVG selection (2,000 genes, Seurat v1 [16]), scaling, PCA (50 components), k-NN graph ($k = 15$), Leiden clustering (resolutions 0.5, 1.0, 1.5) [4], UMAP [5], and Wilcoxon DE testing [17]. The CPU pipeline used Scanpy 1.12 [3] with 100 threads; the GPU pipeline used rapids-singlecell 0.14.1 [8] with CuPy 13.6.0 and the cuGraph Leiden backend. Both pipelines shared the AnnData format [18]. Each step was timed independently; each configuration was repeated five times.

For multi-GPU runs (2, 4, 8 GPUs), preprocessing executed on CPU, PCA and neighbours were distributed across GPUs via Dask-CUDA [19], and Leiden/UMAP/DE executed on a single GPU. Concordance was assessed by HVG Jaccard, PCA loading Spearman rho, kNN Jaccard [20], ARI [21], NMI, and DE log-fold-change correlation. GPU floating-point non-determinism was quantified rather than suppressed [22], [23].

For the stress test, a memory-optimised pipeline moved scaling to CPU, distributed PCA across 8 GPUs via a covariance method, and used lean GPU transfer (only the PCA embedding sent to GPU). A binary search from 3.4M to 13.7M cells identified the failure point. DE was benchmarked separately at 3.4M cells across 41k genes and 81 Leiden clusters, testing Wilcoxon, t-test, and pseudo-bulk aggregation [24] on CPU and GPU.

2.2 Spatial transcriptomics benchmark

Spatial benchmarks used Visium v1 Mouse Brain (2,695 spots), Visium HD 8 um (393,543 bins), and Visium HD 2 um (389,492 bins, subsampled from 6.3M). The CPU baseline used Squidpy 1.8.1 [6]; the GPU pipeline used rapids-singlecell 0.14.1 with spatialdata [25]. Spatial-specific steps included Moran's I, Geary's C, co-occurrence analysis, and neighbourhood enrichment.

2.3 Hardware and software environment

scRNA-seq benchmarks ran on a single DGX H100 node (8 x H100 80 GB, 112 CPU cores, 2 TB RAM) at the UPSCALE/CONVECS cluster, University of Padova. Spatial benchmarks ran on a local workstation (RTX 4090 24 GB, 100 cores, 256 GB RAM). All benchmarks ran inside Singularity containers built from `nvcv.io/nvidia/rapidsai/base:26.02-cuda12-py3.12`.

3 Results

3.1 GPU acceleration achieves up to 120-fold speedup

Table 1 summarises the scRNA-seq benchmark results. At 1.3M cells, the 8-GPU pipeline completed the full analysis in 435 s (7.3 min) vs 52,056 s (14.5 h) on CPU, a 120-fold speedup. Even at 10k cells, single-GPU was 12x faster (3.3 vs 39.5 s). Per-step speedups varied over two orders of magnitude: normalisation reached 329x, HVG selection 24-94x, neighbours 44-88x, and UMAP 27-45x. Data loading was faster on CPU (0.76-0.85x) due to CUDA context initialisation overhead, and Leiden clustering was faster on CPU at 10k cells (0.36x) but crossed over at 50k (1.8x). Figure 1 shows the full per-step speedup landscape across three dataset sizes.

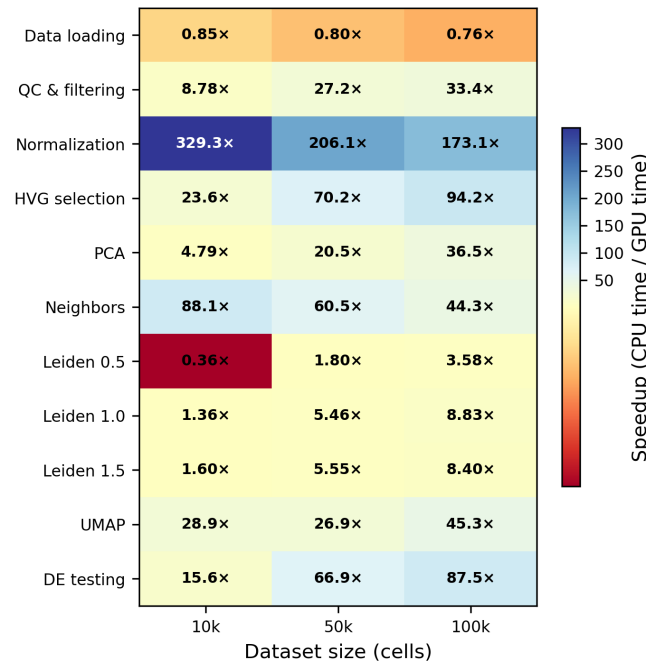


Figure 1. Per-step speedup of single-GPU rapids-singlecell (1xH100 80 GB) relative to CPU Scanpy (100 cores). Values > 1 indicate GPU is faster. Normalisation achieves up to 329x speedup; data loading shows a modest CPU advantage (0.76-0.85x). Leiden clustering crosses over from CPU-faster (0.36x at 10k) to GPU-faster (8.8x at 100k).

Table 1. scRNA-seq benchmark summary. Times are mean +/- SD over five repeats.

Pipeline	Cells	GPU s	Total time (s)	Peak RAM (GB)	Peak VRAM (GB)
CPU	10k	0	39.5 +/- 10.2	1.2	—
GPU	10k	1	3.3 +/- 1.7	2.4	40.7
CPU	50k	0	282.5 +/- 9.1	3.6	—
GPU	50k	1	8.4 +/- 2.2	5.4	40.7
CPU	100k	0	646.7 +/- 10.5	6.7	—
GPU	100k	1	14.8 +/- 2.2	8.8	40.7
CPU	500k	0	4,346.7 +/- 40.3	29.7	—
GPU	500k	2	124.5 +/- 2.9	15.6	52.7
CPU	1.3M	0	52,056.2 +/- 392.0	107.4	—
GPU	1.3M	8	435.2 +/- 16.2	52.3	134.2

3.2 Multi-GPU scaling is sublinear

Adding GPUs beyond two provided diminishing returns. At 1.3M cells, 8 GPUs (435 s) were only 12% faster than 2 GPUs (493 s). In a representative 8-GPU run, CPU preprocessing consumed 75% of wall time, the multi-GPU phase (PCA + neighbours) took 4%, and single-GPU operations (Leiden, UMAP, DE) took 21%. Since only PCA and neighbours benefit from additional GPUs, this is a classic instance of Amdahl's law [26].

3.3 Biological concordance is high

Concordance analysis on 10k cells showed near-identical results. HVG selection was perfectly concordant (Jaccard = 1.0) and PCA loadings were perfectly correlated (Spearman $|\rho| = 1.0$). Leiden clustering yielded ARI = 0.908-0.963 across resolutions, with NMI =

0.951-0.971. DE log-fold-changes showed mean Spearman $\rho = 0.946$ across 40 matched cluster pairs. These concordance levels are consistent with published benchmarks of inter-method variability [27], [28].

3.4 Stress test: 11.9 million cells

The memory-optimised pipeline succeeded at 11.9M cells (119 min, 535 GB RAM, 49 GB aggregate VRAM) and failed at 12.0M cells (Leiden OOM). The binding constraint was CPU-side memory during preprocessing, not GPU VRAM: aggregate VRAM remained flat at 49 GB (7.6% of 640 GB) across all tested scales, while transient CPU memory peaks during sparse-to-dense conversion and Leiden graph construction exceeded the 1.8 TB SLURM allocation at 12M cells. A chunked-preprocessing variant inspired by ScaleSC [11] reduced aggregate VRAM to 10 GB but did not substantially increase the cell limit (12M vs 11.9M), confirming that CPU RAM, not GPU VRAM, is the bottleneck.

3.5 Differential expression at scale

At 3.4M cells, pseudo-bulk aggregation completed in 128 s (44x faster than cell-level CPU t-test at 5,598 s). GPU Wilcoxon (826 s) outperformed GPU t-test (1,656 s), and multi-GPU scatter was equivalent to single-GPU chunk-and-stream, indicating I/O-bound rather than compute-bound behaviour. Pseudo-bulk is simultaneously the fastest and the most statistically rigorous approach, as it avoids pseudoreplication in multi-donor experiments [24].

3.6 Spatial transcriptomics

We benchmarked three 10x Visium platforms on a local RTX 4090. End-to-end speedup ranged from 1.7x on Visium v1 (2,695 spots, 10.0 s CPU vs 6.0 s GPU) to 51.6x on Visium HD 8 um (393,543 bins, 4,068 s CPU vs 79 s GPU) and 10.8x on Visium HD 2 um (389,492 bins, 1,042 s vs 97 s). The dominant contributor to the HD 8 um speedup was co-occurrence analysis, which achieved a 3,272x speedup (3,573 s to 1.09 s) by parallelising the $O(n^2)$ CPU computation across GPU threads. Spatial autocorrelation concordance was near-perfect (Moran's I and Geary's C Spearman $\rho \geq 0.9995$) and top-50 spatially variable genes were identical (Jaccard = 1.0). Clustering concordance was high on Visium v1 (ARI = 0.856) and HD 8 um (ARI = 0.974) but diverged on HD 2 um (ARI = 0.080) due to algorithmic differences between the leidenalg and cuGraph implementations at high resolution with many near-degenerate partitions.

4 Conclusions

GPU-accelerated single-cell analysis via rapids-singlecell on NVIDIA H100 GPUs reduces a 14.5-hour CPU pipeline to 7.3 minutes at 1.3M cells, with biologically concordant results. The heterogeneous per-step speedup landscape (0.76x for data loading to 329x for normalisation) reflects fundamental differences in computational structure: trivially parallelisable element-wise operations achieve the highest speedups, while data loading is dominated by fixed CUDA context overhead and Leiden clustering on small graphs is limited by low arithmetic intensity, consistent with the roofline model [29]. Multi-GPU scaling is sublinear (12% improvement from 2 to 8 GPUs), a direct consequence of Amdahl's law: only 4% of wall time runs on multiple GPUs. A practical consequence is that a DGX H100 node is more efficiently used as eight independent single-GPU workers running concurrent analyses than as an 8-way worker for one analysis. Most of the practical value is accessible on a single cloud-rented H100 at approximately USD 2-4 per hour. The principal scalability bottleneck is CPU-side preprocessing, not GPU VRAM: at 11.9M cells, aggregate VRAM was 7.6% of capacity while CPU RAM reached 535 GB. The

capacity limit is a function of cells x HVGs: with 2,000 HVGs, linear extrapolation suggests 20M cells at 1,000 HVGs and 4M at 5,000 HVGs. For differential expression, pseudo-bulk aggregation is simultaneously the fastest (44x vs t-test) and most statistically rigorous approach [24].

For spatial transcriptomics, GPU acceleration delivers 51.6x end-to-end speedup on Visium HD, with co-occurrence analysis achieving a 3,272x speedup and spatial autocorrelation concordance exceeding 0.9995. GPU-native implementations for the remaining CPU-only spatial operations represent opportunities for future work.

Our study has several limitations: scRNA-seq concordance was measured at 10k cells; spatial benchmarks ran on a local RTX 4090; we did not benchmark integration methods or multi-node configurations. These results provide practical guidance for the computational bioinformatics community as single-cell and spatial datasets grow toward atlas scale [1].

Conflict of Interest

None

References

- [1] A. Regev *et al.*, “The Human Cell Atlas,” *eLife*, vol. 6, p. e27041, 2017, doi: 10.7554/eLife.27041.
- [2] Z. Yao *et al.*, “A high-resolution transcriptomic and spatial atlas of cell types in the whole mouse brain,” *Nature*, vol. 624, no. 7991, pp. 317–332, 2023, doi: 10.1038/s41586-023-06812-z.
- [3] F. A. Wolf, P. Angerer, and F. J. Theis, “SCANPY: Large-scale single-cell gene expression data analysis,” *Genome Biology*, vol. 19, no. 1, p. 15, 2018, doi: 10.1186/s13059-017-1382-0.
- [4] V. A. Traag, L. Waltman, and N. J. van Eck, “From Louvain to Leiden: Guaranteeing well-connected communities,” *Scientific Reports*, vol. 9, p. 5233, 2019, doi: 10.1038/s41598-019-41695-z.
- [5] L. McInnes, J. Healy, N. Saul, and L. Großberger, “UMAP: Uniform manifold approximation and projection,” *Journal of Open Source Software*, vol. 3, no. 29, p. 861, 2018, doi: 10.21105/joss.00861.
- [6] G. Palla *et al.*, “Squidpy: A scalable framework for spatial omics analysis,” *Nature Methods*, vol. 19, pp. 171–178, 2022, doi: 10.1038/s41592-021-01358-2.
- [7] RAPIDS Development Team, “RAPIDS: Open GPU data science.” 2024. Available: <https://rapids.ai>
- [8] S. Dicks *et al.*, “GPU-accelerated single-cell analysis at scale with rapids-singlecell,” *arXiv preprint arXiv:2603.02402*, 2026, doi: 10.48550/arXiv.2603.02402.
- [9] A. Taylor-Weiner *et al.*, “Scaling computational genomics to millions of individuals with GPUs,” *Genome Biology*, vol. 20, p. 228, 2019, doi: 10.1186/s13059-019-1836-7.
- [10] C. Nolet *et al.*, “Accelerating single-cell genomic analysis with GPUs,” *bioRxiv*, 2022, doi: 10.1101/2022.05.26.493607.
- [11] W. Hu *et al.*, “ScaleSC: A superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU,” *Bioinformatics Advances*, vol. 5, no. 1, p. vbaf167, 2025, doi: 10.1093/bioadv/vbaf167.
- [12] C. Gardner, S. Jeong, O. Khatavkar, A. Moon, Q. Cao, and T.-H. Ahn, “Evaluating accuracy and performance tradeoffs in GPU accelerated single cell RNA-seq analysis,” in *Proceedings of the SC ’25 workshops of the international conference for high performance computing, networking, storage and analysis*, ACM, 2025. doi: 10.1145/3731599.3767378.

- [13] G. X. Y. Zheng *et al.*, “Massively parallel digital transcriptional profiling of single cells,” *Nature Communications*, vol. 8, p. 14049, 2017, doi: 10.1038/ncomms14049.
- [14] M. D. Luecken and F. J. Theis, “Current best practices in single-cell RNA-seq analysis: A tutorial,” *Molecular Systems Biology*, vol. 15, no. 6, p. e8746, 2019, doi: 10.15252/msb.20188746.
- [15] L. Heumos *et al.*, “Best practices for single-cell analysis across modalities,” *Nature Reviews Genetics*, vol. 24, no. 8, pp. 550–572, 2023, doi: 10.1038/s41576-023-00586-w.
- [16] R. Satija, J. A. Farrell, D. Gennert, A. F. Schier, and A. Regev, “Spatial reconstruction of single-cell gene expression data,” *Nature Biotechnology*, vol. 33, no. 5, pp. 495–502, 2015, doi: 10.1038/nbt.3192.
- [17] C. Soneson and M. D. Robinson, “Bias, robustness and scalability in single-cell differential expression analysis,” *Nature Methods*, vol. 15, no. 4, pp. 255–261, 2018, doi: 10.1038/nmeth.4612.
- [18] I. Virshup, S. Rybakov, F. J. Theis, P. Angerer, and F. A. Wolf, “Anndata: Access and store annotated data matrices,” *Journal of Open Source Software*, vol. 9, no. 101, p. 4371, 2024, doi: 10.21105/joss.04371.
- [19] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” in *Proceedings of the 14th python in science conference*, K. Huff and J. Bergstra, Eds., 2015, pp. 126–132. doi: 10.25080/Majora-7b98e3ed-013.
- [20] M. D. Luecken *et al.*, “Benchmarking atlas-level data integration in single-cell genomics,” *Nature Methods*, vol. 19, no. 1, pp. 41–50, 2022, doi: 10.1038/s41592-021-01336-8.
- [21] L. Hubert and P. Arabie, “Comparing partitions,” *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985, doi: 10.1007/BF01908075.
- [22] S. Shanmugavelu, M. Taillefumier, C. Culver, O. Hernandez, M. Coletti, and A. Sedova, “Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications,” in *Proceedings of the SC ’24 workshops of the international conference on high performance computing, networking, storage, and analysis (CORRECTNESS 2024)*, 2024. doi: 10.1109/SCW63240.2024.00028.
- [23] C. Collange, D. Defour, S. Graillat, and R. Iakymchuk, “Numerical reproducibility for the parallel reduction on multi- and many-core architectures,” *Parallel Computing*, vol. 49, pp. 83–97, 2015, doi: 10.1016/j.parco.2015.09.001.
- [24] J. W. Squair *et al.*, “Confronting false discoveries in single-cell differential expression,” *Nature Communications*, vol. 12, p. 5692, 2021, doi: 10.1038/s41467-021-25960-2.
- [25] L. Marconato *et al.*, “SpatialData: An open and universal data framework for spatial omics,” *Nature Methods*, vol. 22, pp. 58–62, 2024, doi: 10.1038/s41592-024-02212-x.
- [26] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the april 18–20, 1967, spring joint computer conference (AFIPS ’67)*, 1967, pp. 483–485. doi: 10.1145/1465482.1465560.
- [27] L. Tian *et al.*, “Benchmarking single cell RNA-sequencing analysis pipelines using mixture control experiments,” *Nature Methods*, vol. 16, pp. 479–487, 2019, doi: 10.1038/s41592-019-0425-8.
- [28] A. Duò, M. D. Robinson, and C. Soneson, “A systematic performance evaluation of clustering methods for single-cell RNA-seq data,” *F1000Research*, vol. 7, p. 1141, 2020, doi: 10.12688/f1000research.15666.3.
- [29] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009, doi: 10.1145/1498765.1498785.